

Configure Prometheus for DCGM Exporter

DCGM Exporter serves Prometheus metrics from `/metrics` on port `9400` by default. Complete [Install DCGM Exporter](#) and verify the exporter endpoint before adding it to Prometheus.

Add a Static Scrape Target

Verify that Prometheus can reach the exporter from its own network:

```
$ curl --fail http://gpu-node-1:9400/metrics
```

Add each reachable GPU host to `prometheus.yml`:

```
scrape_configs:
- job_name: dcm-exporter
  scrape_interval: 30s
  scrape_timeout: 25s
  static_configs:
    - targets:
      - gpu-node-1:9400
      - gpu-node-2:9400
```

Reload or restart Prometheus according to its deployment method. See the [Prometheus configuration reference](#) for service discovery, relabeling, authentication, and other scrape settings.

Align Collection and Scrape Intervals

DCGM collection, exporter serving, and Prometheus scraping have separate timing controls:

- The exporter collection interval and YAML watch-group intervals request how often DCGM refreshes cached samples.
- `scrape_interval` controls how often Prometheus reads `/metrics`.
- `scrape_timeout` limits one Prometheus read.
- The exporter `--web-write-timeout` limits how long it can write an HTTP response.

Keep `scrape_timeout` below both `scrape_interval` and the exporter `--web-write-timeout`. A scrape does not force DCGM to query the hardware, so scraping more frequently than the effective DCGM watch cadence can return the same cached sample more than once.

Use a ServiceMonitor

The DCGM Exporter Helm chart enables a `ServiceMonitor` by default. The cluster must have the Prometheus Operator custom resource definitions. The chart resource selects the chart-managed Service. The Prometheus resource's `ServiceMonitor` and namespace selectors must, in turn, admit that `ServiceMonitor` and its namespace. If Prometheus selects `ServiceMonitor` objects by label, set the required labels with `serviceMonitor.additionalLabels`.

Set an explicit scrape budget with chart values:

```
service:
  webReadTimeout: 10s
  webWriteTimeout: 30s

serviceMonitor:
  enabled: true
  interval: 30s
  scrapeTimeout: 25s
```

The chart-managed `ServiceMonitor` assumes unauthenticated HTTP. If the exporter uses TLS or basic authentication, disable that resource and provide a Prometheus scrape configuration with matching credentials and TLS settings. See the [Prometheus Operator getting started guide](#) and the [exporter-toolkit web configuration](#).

Verify the resource:

```
$ kubectl get servicemonitor \
  --namespace gpu-monitoring \
  dcgm-exporter
```

Verify the Prometheus Target

Open the Prometheus targets page and confirm that the `dcgm-exporter` target is `UP`. Use these queries as smoke tests:

```
up{job="dcgm-exporter"}
DCGM_FI_DEV_GPU_UTIL
```

The `up` query verifies the scrape. The DCGM metric verifies that the target returns GPU telemetry selected by the collector configuration. See [Querying Prometheus](#) for PromQL syntax and the [Prometheus HTTP API](#) for automated verification.

Next Steps

- [Install DCGM Exporter](#) covers installation, exporter configuration, endpoint verification, and administration.

- [DCGM Exporter Metrics](#) documents the shipped metric inventory, exporter-owned metrics, and labels.
- [dcgm-exporter](#) provides exact collection, HTTP, authentication, and logging settings.
- [Fields and Watches](#) explains DCGM collection cadence, retained samples, and cached values.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.